

GTX 1050 Mobile

Passthrough Optimus su Proxmox

Relazione tecnica: diagnosi, recupero della VBIOS OEM, SSDT ACPI dinamica, switch idempotente tra VM Linux, Secure Boot e benchmark.

Componente	Esito
Portatile	HP Pavilion Laptop 15-cs1xxx, scheda madre HP 856A
Host	Proxmox VE 9.1.5, kernel 6.17.9, IOMMU e VFIO attivi
GPU	NVIDIA GP107M GTX 1050 Mobile (Pascal), PCI ID 10de:1c8d
Firmware	VBIOS OEM HP inclusa nel repository privato, 169472 byte, 86.07.5F.00.2C
Ubuntu	Driver NVIDIA 580.173.02, 4096 MiB, nvidia-smi valido
Kali	SeaBIOS/Q35; switch reale Ubuntu -> Kali -> Ubuntu riuscito
Switch VM	Cleanup, discovery PCI/ACPI e SSDT riusciti in entrambe le direzioni
Prova rendering	glxgears sul display NVIDIA :2, circa 24-25 mila FPS

Repository: proxmox-gtx1050-mobile-passthrough

1. Problema e criterio di successo

Una GPU mobile Optimus non è una GPU desktop isolata: nel Pavilion 15-cs1xxx il display fisico resta normalmente collegato alla iGPU e il driver NVIDIA può richiedere la propria VBIOS attraverso ACPI. hostpci assegna correttamente il dispositivo PCI, ma non aggiunge automaticamente un metodo ACPI _ROM: per questo il passthrough standard con una ROM PCI non era sufficiente.

La GTX 1050 Mobile di questo caso usa GP107M, una GPU Pascal laptop con 640 CUDA core nella configurazione GTX 1050 e 4 GiB verificati nel guest. In Optimus la NVIDIA è render-only: può accelerare CUDA/OpenGL/Vulkan, ma non ha necessariamente un CRTC o un connettore display assegnabile. La procedura è quindi mirata a NVIDIA mobile/Optimus e richiede una VBIOS OEM coerente; non è una soluzione universale per ogni GPU laptop.

Il criterio di successo non è solo vedere una riga in lspci. La GPU deve essere assegnabile a una VM, il driver proprietario deve caricare, nvidia-smi deve riportare nome, driver e memoria, e un processo grafico deve usare realmente la GPU.

2. Termini fondamentali

BDF	Indirizzo PCI dominio:bus:device.funzione. La GPU host è 0000:02:00.0.
VBIOS / ROM	Firmware della GPU; in questo caso il file .rom contiene la VBIOS OEM.
IOMMU e VFIO	Isolano DMA e assegnano il dispositivo reale a QEMU invece che al driver host.
Optimus muxless	La iGPU guida il pannello; NVIDIA è render-only e può dipendere dai metodi ACPI della piattaforma.
ACPI	Descrizione firmware dell'hardware e dei metodi richiamati dal sistema operativo.
ASL / AML / SSDT	ASL è testo, AML è bytecode compilato, SSDT è tabella ACPI aggiuntiva.
_ROM	Metodo ACPI che fornisce una porzione della ROM al driver, dati offset e dimensione.
fw_cfg	Canale QEMU per fornire piccoli blob al guest; qui trasporta la VBIOS.
MOK	Machine Owner Key: certificato da registrare prima del kernel per fidare moduli DKMS con Secure Boot.
OVMF / Q35 / QGA	UEFI QEMU, chipset PCIe virtuale e Guest Agent usato per discovery e verifica nel guest.

3. Perché la soluzione ACPI era necessaria

Il solo romfile espone una ROM nella configurazione PCI virtuale. Il driver della GTX 1050 Mobile, nel percorso Optimus, cerca invece _ROM nel device ACPI. Per questo rombar=1 mostrava byte ROM ma non risolveva l'inizializzazione del driver. VBIOS e file .rom sono qui lo stesso firmware; rombar e invece una finestra PCI virtuale, non il metodo ACPI.

La soluzione passa lo stesso file VBIOS OEM mediante fw_cfg. Una SSDT in AML si aggancia al device della GPU, legge il file una volta, lo mantiene in un buffer e implementa _ROM restituendo il segmento richiesto. rombar=0 resta intenzionale.

gtx1050_hp_native.rom sul nodo

```
      |
      v
QEMU fw_cfg -> SSDT AML (buffer FWBI) -> _ROM(offset,length) -> driver NVIDIA
0000:02:00.0
|   |   |   +-- funzione 0: GPU
|   |   +----- dispositivo 00
|   +----- bus 02
+----- dominio PCI 0000
```

4. PCI -> ACPI senza valori fissi

Il guest restituisce la catena PCI con lspci -PP. Ogni hop viene convertito nel nome ACPI Sxx: valore = slot * 8 + funzione.

```
00:1c.0/01:00.0
1c.0 -> 0x1c * 8 + 0 = 0xe0 -> SE0
00.0 -> 0x00 * 8 + 0 = 0x00 -> S00
Scope finale: \_SB.PCI0.SE0.S00
```

ASL e il testo generato; iasl lo compila in AML, il bytecode che QEMU carica. External dichiara il device già presente nella DSDT, Scope lo apre, RINT legge fw_cfg nel buffer FWBI e _ROM restituisce Mid(FWBI, offset, length). Lo script avvia brevemente la VM, scopre questa topologia reale e genera la SSDT specifica della VM. Così il metodo può adattarsi a un'altra NVIDIA mobile, se vengono indicati BDF host e VBIOS OEM corretti.

4b. SSDT: lettura guidata

```
External (\_SB.PCI0.SE0.S00, DeviceObj) # device esistente
Scope (\_SB.PCI0.SE0.S00) # aggiungi metodi alla GPU
Name (FWIT, 0) / Name (FWBI, Buffer({})) # flag + buffer VBIOS
OperationRegion (... 0x510, 2) # porte QEMU fw_cfg
FISL (...) # trova nome e dimensione blob
RINT () # carica una volta FWBI
_ROM (offset,length) -> Mid(FWBI,...) # risposta al driver
```

RWRD, RDWD e RBUF sono lettori rispettivamente a 16 bit, 32 bit e buffer; Serialized evita letture concorrenti. Il walkthrough del repository spiega ogni blocco e anche il limite a 4 KiB per richiesta _ROM.

5. Switch GPU idempotente

- Prima del primo switch: --prepare-host installa acpica-tools/pciutils, installa la ROM OEM se diversa, aggiunge solo flag IOMMU/VFIO mancanti e aggiorna initramfs; il reboot e sempre esplicito.
- Menu numerato delle VM oppure --vm VMID per automazione.
- Ricerca proprietario corrente della GPU e arresto pulito della sola VM coinvolta.
- Cleanup mirato: hostpci, romfile, rombar, argomenti SSDT/fw_cfg e CPU hidden generati dallo script.
- Riavvio automatico della VM sorgente che prima era accesa.
- Discovery ACPI, generazione AML, avvio finale e verifica nvidia-smi.
- Nessuna conversione forzata di BIOS, OVMF, SeaBIOS, Q35 o vga: questi restano proprietà della VM.
- Se hostpci, SSDT/fw_cfg e nvidia-smi sono già corretti nella VM richiesta, non opera né riavvia; se resta solo MOK, non ricostruisce lo switch.
- Test reale completato: Ubuntu 1001 OVMF/Q35 -> Kali 1000 SeaBIOS/Q35 -> Ubuntu 1001; cleanup e SSDT sono stati rigenerati in entrambe le direzioni.

Comandi principali

```
gpu-vm-switch --prepare-host --rom-source ./firmware/gtx1050_hp_native.rom --yes
gpu-vm-switch --prepare-host --rom-source ./firmware/gtx1050_hp_native.rom --reboot --yes
gpu-vm-switch
gpu-vm-switch --vm 1001 --yes
gpu-vm-switch --vm 1001 --dry-run --yes
gpu-vm-switch --gpu 0000:03:00 --rom /usr/share/kvm/oem.rom --vm 123 --yes
gpu-vm-switch --vm 123 --skip-drivers --yes
```

Driver automatici: Ubuntu, Debian, Kali, Arch, Fedora, RHEL, Rocky e AlmaLinux. Per Debian la sorgente contrib non-free non-free-firmware e aggiunta solo se non è già presente; la logica gestisce anche il formato Deb822. Il BIOS del laptop resta l'unico prerequisito non automatizzabile: VT-d/AMD-Vi deve essere attivo prima del boot Proxmox.

5b. Procedura riproducibile e criteri di prova

```
# nodo: prima di applicare
sha256sum firmware/gtx1050_hp_native.rom
lspci -nnk -s 0000:02:00.0
gpu-vm-switch --prepare-host --rom-source ./firmware/gtx1050_hp_native.rom --dry-run --yes

# nodo: applicazione e controllo dopo reboot
gpu-vm-switch --prepare-host --rom-source ./firmware/gtx1050_hp_native.rom --yes
cat /proc/cmdline; lspci -nnk -s 0000:02:00.0; gpu-vm-switch --self-test

# switch: simulazione, applicazione, prova driver
gpu-vm-switch --vm 1001 --dry-run --yes
gpu-vm-switch --vm 1001 --yes
qm guest exec 1001 -- /usr/bin/nvidia-smi --query-gpu=name,driver_version,memory.total --format=csv,noheader
```

La sequenza è deliberatamente divisa in inventario, dry-run, applicazione e osservazione. Il risultato positivo richiede: hash ROM atteso, IOMMU e vfio-pci dopo reboot, self-test AML riuscito, hostpci con args SSDT/fw_cfg e nvidia-smi con exitcode 0. nvidia-glxgears e nvidia-smi provano poi il rendering. Il runbook del repository aggiunge comandi esatti per QEMU Guest Agent, Secure Boot/MOK, prova Ubuntu -> Kali -> Ubuntu e condizioni in cui fermarsi; il validatore documentale non viene spacciato per prova hardware.

6. Secure Boot senza automazione fittizia

MOK non e causato dal nome NVIDIA: compare quando DKMS compila il modulo localmente e Secure Boot richiede una firma di una chiave fidata. DKMS puo firmare con una propria chiave, ma questa deve essere registrata come Machine Owner Key. La schermata MOK e fuori dall'OS: SSH e QEMU Guest Agent non possono premere tasti prima dell'avvio.

Lo script rileva lo stato reale nel guest, con mokutil oppure con la variabile EFI SecureBoot. Per mantenere Secure Boot esiste --mok-manual: non finge di premere il firmware, conserva la GPU gia configurata e stampa certificati/passaggi per noVNC. Dopo sudo mokutil --import certificato.der, al riavvio l'utente sceglie Enroll MOK, Continue, Yes e inserisce la password temporanea.

```
gpu-vm-switch --vm 123 --mok-manual  
gpu-vm-switch --vm 123 --disable-secure-boot --yes
```

L'alternativa Secure Boot off e permanente e solo OVMF/efidisk0 4m. Prima crea EFI/BOOT/BOOTX64.EFI, copia le vecchie variabili EFI e metadata in /usr/share/kvm/optimus-gpu-switch/efi-backups/, poi sostituisce efidisk0 con il template senza chiavi. Proxmox conserva il volume precedente come unused disk. Il rollback resta attivo fino al primo boot con Guest Agent; il cambio non e stato eseguito sulla Ubuntu principale, quindi prima dell'uso fare snapshot e tenere noVNC disponibile. Questo evita MOK ma abbassa la protezione della catena di boot.

7. Benchmark e prova di rendering

glmark2 da SSH ha fallito con Could not initialize canvas: e previsto, perche glmark2 X11 necessita un display. Il DRM della GPU mobile render-only non offre un CRTC utile al test. E stato quindi creato un Xorg NVIDIA headless sul display :2 e installato il wrapper nvidia-glxgears.

```
nvidia-glxgears
120907 frames in 5.0 seconds = 24181.367 FPS
125006 frames in 5.0 seconds = 25001.096 FPS
watch -n 1 nvidia-smi
nvidia-smi
```

htop non visualizza i contatori NVIDIA perche legge CPU e RAM del sistema operativo. nvidia-smi e nvidia-smi interrogano invece driver e GPU.

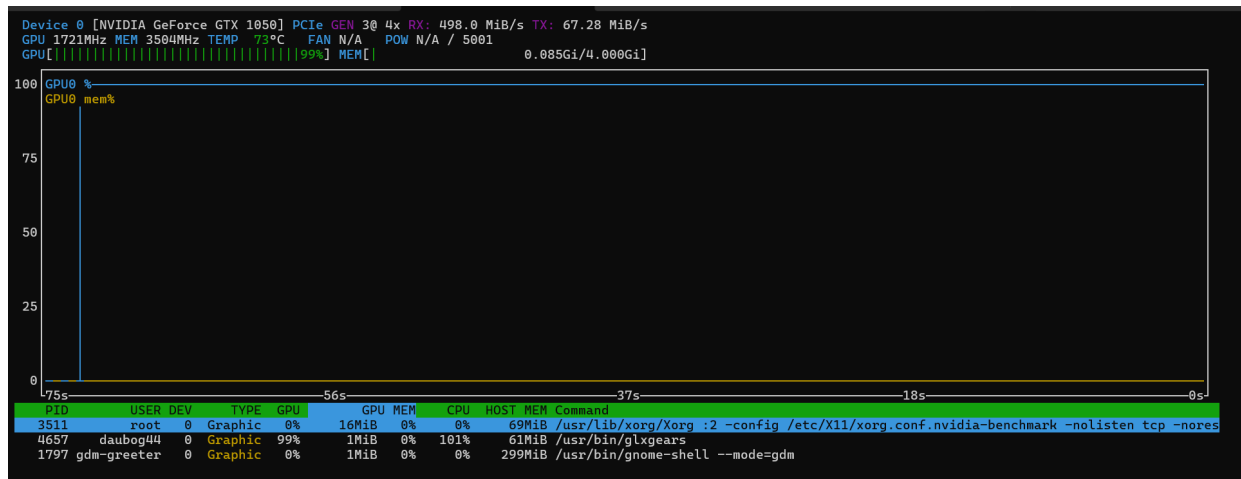


Figura 1 - nvidia-smi: glxgears usa il 99% della GPU; Xorg gira su :2.

8. Estrazione della VBIOS OEM

Il punto di partenza è il pacchetto driver/firmware ufficiale HP per il Pavilion 15-cs1xxx, estratto localmente fino al payload, per esempio 084C0.bin. Lo script Python non scarica nulla: cerca la signature PCI option ROM 55 aa, verifica PCIR, vendor NVIDIA e device 1c8d. Scansiona il payload RAW e stream LZMA, quindi estrae tutte le immagini della ROM fino al flag finale.

```
python3 scripts/extract_gtx1050_rom.py /tmp/084C0.bin \
--device-id 1c8d \
--output /usr/share/kvm/gtx1050_hp_native.rom
```

Per rendere riproducibile questo laboratorio privato, la ROM estratta è inclusa come firmware/gtx1050_hp_native.rom: 169472 byte, VBIOS 86.07.5F.00.2C, SHA-256 33abd3bc3f658b0536da0617a76076b56e5af124701271211d6d127cda22c322. E firmware OEM, non universale: verificare licenza e usare --force solo dopo confronto di vendor, device ID e origine del payload.

9. Tentativi che non hanno risolto il problema

Tentativo	Perche falliva	Correzione
Solo romfile / rombar	Il driver Optimus non leggeva la ROM dalla sola finestra PCI.	SSDT_ROM + fw_cfg.
ACPI scritto a mano	Cambiando bridge cambia lo scope del device.	Discovery lspci -PP e conversione Sxx.
glmark2 via SSH	Mancava un canvas X11.	Xorg NVIDIA :2 e glxgears.
Prima installazione Kali	Headers/DKMS non pronti.	Headers, build tools, DKMS e riavvio.
Repo Debian riscritta	Automazione non idempotente.	Verifica componenti prima di aggiungere.

10. Ripetere il procedimento

- Controllare che IOMMU/VFIO isolino GPU e funzione audio sul nodo.
- Estrarre la VBIOS OEM del nuovo dispositivo e verificare BDF e PCI ID con lspci.
- Configurare la VM con Q35 e Guest Agent; mantenere il firmware scelto dalla VM.
- Eseguire gpu-vm-switch con --gpu e --rom, poi nvidia-smi.
- Per un test grafico su guest muxless, creare un display X NVIDIA headless e usare nvidia-glxgears.
- Scegliere consapevolmente MOK o --disable-secure-boot per OVMF; non alternare automaticamente le due politiche.

Riferimenti

QEMU fw_cfg: https://qemu-project.gitlab.io/qemu/specs/fw_cfg.html

ACPI: <https://uefi.org/acpi/specs>

Proxmox VE Administration Guide: <https://pve.proxmox.com/pve-docs/pve-admin-guide.html>

NVIDIA driver kernel modules: <https://docs.nvidia.com/datacenter/tesla/driver-installation-guide/kernel-modules.html>

NVIDIA GTX 1050 laptop / Pascal: <https://www.nvidia.com/en-us/geforce/news/nvidia-geforce-gtx-1050-laptops/>

Linux kernel VFIO: <https://docs.kernel.org/driver-api/vfio.html>

ACPICA / iasl: <https://acpica.org/>